

POWER AI BI ASSISTANT

A Premium AI-Driven Business Intelligence Platform

Document Type:	Technical & System Architecture Report
Prepared For:	Academic & Project Submission
Author Account:	hariraman-24
GitHub Project:	github.com/hariraman-24/ai-bi-bkups
Current Status:	Live & Operational

1. Executive Summary & Project Objectives

The Power AI BI Assistant is a state-of-the-art Business Intelligence application designed to make complex relational databases and corporate data files accessible via natural language. Historically, business leaders have been forced to rely on dedicated data analysts to write SQL or run spreadsheet macros to produce sales performance metrics. This application changes that paradigm by translating English questions into executable queries, mathematical models, and graphic charts on the fly.

By utilizing a Model Context Protocol (MCP) server structure and local LLM/OpenAI APIs, the assistant provides accurate analytical responses, forecasts sales trends, parses document structures (including PDFs, Excel, and CSVs), and automatically drafts human-readable explanations of SQL execution datasets. The platform contains dual user interfaces—a visual dashboard in Streamlit and a web client in React and TypeScript—catering to varied end-user preferences.

Core Project Objectives:

- **Natural Language Access (NLP):** Democratize querying by enabling standard users to ask questions like *'Who was the top employee in January 2025 by sales?'* and get instant answers.
- **Dual Frontend Delivery:** Offer a dark-mode Streamlit analytical page and a reactive single-page React chat application.
- **Intelligent Decision Routing:** Classify query intent to ensure queries for datasets, charts, or mathematical forecasting are routed to their designated specialized engine.
- **High-Fidelity Visualizations:** Instantly build corresponding data visualizations matching a clean corporate aesthetic.

2. System Architecture & Technical Flow

The architecture is structured with modular backend engines exposing endpoints through an exposition layer, accessed by independent frontend clients. Below is a structural mapping of a user query's execution path:

Step	Component	Function & Output
1. Request	Streamlit Client / React Interface	User types natural language query or uploads a file.
2. Classification	Intent Engine (Ollama / OpenAI fallback)	Parses query semantic tokens and determines if it is a general chat, database query, forecasting run, or visual chart.
3. Routing	Decision Engine (backend/decision_engine.py)	Dispatches execution payload to the database SQL generator, the forecasting engine, or the file server.
4. Synthesis	SQL Generator & Validator	Generates dialect-safe SQL (Postgres/MySQL) and validates database operations against schema rules before executing.
5. Insight Engine	Narrative Generator (backend/insight_engine.py)	Transforms raw data columns into natural language insights describing peaks, valleys, and summaries.
6. Response	Visualization & Charting Engine	Generates a clean visual plot (Matplotlib/Plotly) and displays the analytical report to the user.

3. Core Backend Engines

3.1. Intent Engine & Decision Router

Implemented in `backend/intent_engine.py` and `backend/decision_engine.py`, this layer serves as the primary dispatcher. It combines lightweight keyword pattern heuristics with LLM semantic classifiers. If a query matches forecasting criteria (e.g., contains 'forecast', 'predict'), the request is routed to the forecasting module. If a chart is requested, it ensures parameters are prepared for the visualization module.

3.2. SQL Generator & Query Validator

Responsible for generating robust SQL statements. The generator in `backend/sql_generator.py` references schema contexts and generates database-specific queries. Before execution, the generated query is run through `backend/query_validator.py`. The validator checks for syntax irregularities, schema matches, and prevents potentially destructive queries (e.g., DROP, DELETE, or UPDATE commands), acting as a security gatekeeper.

3.3. Sales & Predictive Forecasting Engine

When forecasting intent is identified, the application calls `backend/forecasting/forecast.py`. This module reads historical sales volumes from the database, builds seasonal time-series models (Holt-Winters or Linear Regressions depending on dataset characteristics), and generates a predicted sequence of sales for the subsequent period, returning both statistical data points and confidence metrics.

3.4. Dynamic Insight Engine

Found in `backend/insight_engine.py`, this component runs after the dataset is fetched from the database. Rather than forcing the user to interpret a raw table of data, the Insight Engine analyzes trends, detects peak sales months, highlights employees with the highest values, and summarizes key insights into a readable prose paragraphs.

4. Database Schema & Project Structure

The relational database schema models a typical company sales tracking environment. The script `database/schema.sql` seeds these four main tables:

Table	Primary Key	Key Attributes	Description
products	product_id	product_name, category, price	Catalog of products sold by the company.
customers	customer_id	customer_name, region	Demographics and locations of clients.
employees	employee_id	employee_name, department	Internal staff listing.
sales	sale_id	product_id, customer_id, employee_id, sales_amount, sale_date	Main transactions log linking all tables.

Codebase Layout:

backend/	Contains core Python engines (SQL generator, forecasting, intents, database connection handlers).
frontend/	Contains the Streamlit web dashboard app (<code>app.py</code>) and the React/Vite single page source files.
database/	Contains the relational schema DDL files.
static/	Stores visualization outputs and mockup image files.
visualization/	Contains graphic plot helpers (<code>charts.py</code>).

5. Client Interfaces & UI Design

The Power AI BI platform delivers dual user experiences, catering to both structured analytics and fluid conversation patterns.

5.1. Streamlit Executive Dashboard

Implemented in frontend/app.py, the Streamlit app delivers an executive-level summary page. It features styled dark-slate gradients, interactive database connector modals, quick-click metric KPI cards that translate sales queries into visual KPIs, and full file-upload integrations. The interface is optimized to automatically adjust layouts depending on screens.



Figure 1: Generated mockup representing the dark-mode BI system dashboard interface.

5.2. React + TypeScript Web App

For a responsive, standalone chat client, the codebase holds a full Single Page Application in frontend/src/. It connects to the backend endpoints, offering a modern, clean sidebar workspace, chat timelines, and custom message components that display formatted charts and tables.

6. Setup, Run & Verification Guide

Follow these steps to run the application components locally in your development workspace.

Step 1: Relational Database Initialization

Import the schema DDL inside your local database client. For example, in MySQL:

```
mysql -u root -p < database/schema.sql
```

Step 2: Initialize the Backend & Streamlit App

Create a virtual environment and run the dashboard directly:

```
# Create venv python -m venv venv # Activate venv (Windows) venv\Scripts\activate #  
Install dependencies pip install -r backend/requirements.txt # Start Streamlit app  
streamlit run frontend/app.py
```

Step 3: Run the React Web App Client

Start the frontend development server using Node.js npm utilities:

```
cd frontend npm install npm run dev
```

Conclusion

The Power AI BI Assistant succeeds in wrapping advanced analytical processes in an intuitive interface. Users can execute secure queries, observe trends, generate automated summaries, and download reports interactively. All code has been pushed and verified on the GitHub master repository at github.com/hariraman-24/ai-bi-bkups.